Rich Descriptive Models for Reusable ROS Software Components

Riddhesh P. More

Supervised by: Prof. Sebastian Houben, Dr. Björn Kahl
Bonn-Rhein-Sieg University of Applied Sciences

hbrs.png

event.jpg

November 17, 2025

Presentation Outline

WHY: The "Semantic Gap"

- **The Problem:** Finding the right ROS (Robot Operating System) package is difficult.
- Developers rely on keyword searches (e.g., "path planning", "slam").
- This fails to capture:
  - True functionality (e.g., "motion control" vs. "path planning")
  - Hardware requirements (e.g., "LiDAR", "IMU")
  - Package compatibility
- **The "Why":** This "semantic gap" between human intent and machine data is a critical bottleneck in robotics development.

Motivating Context: IAM4RAIL

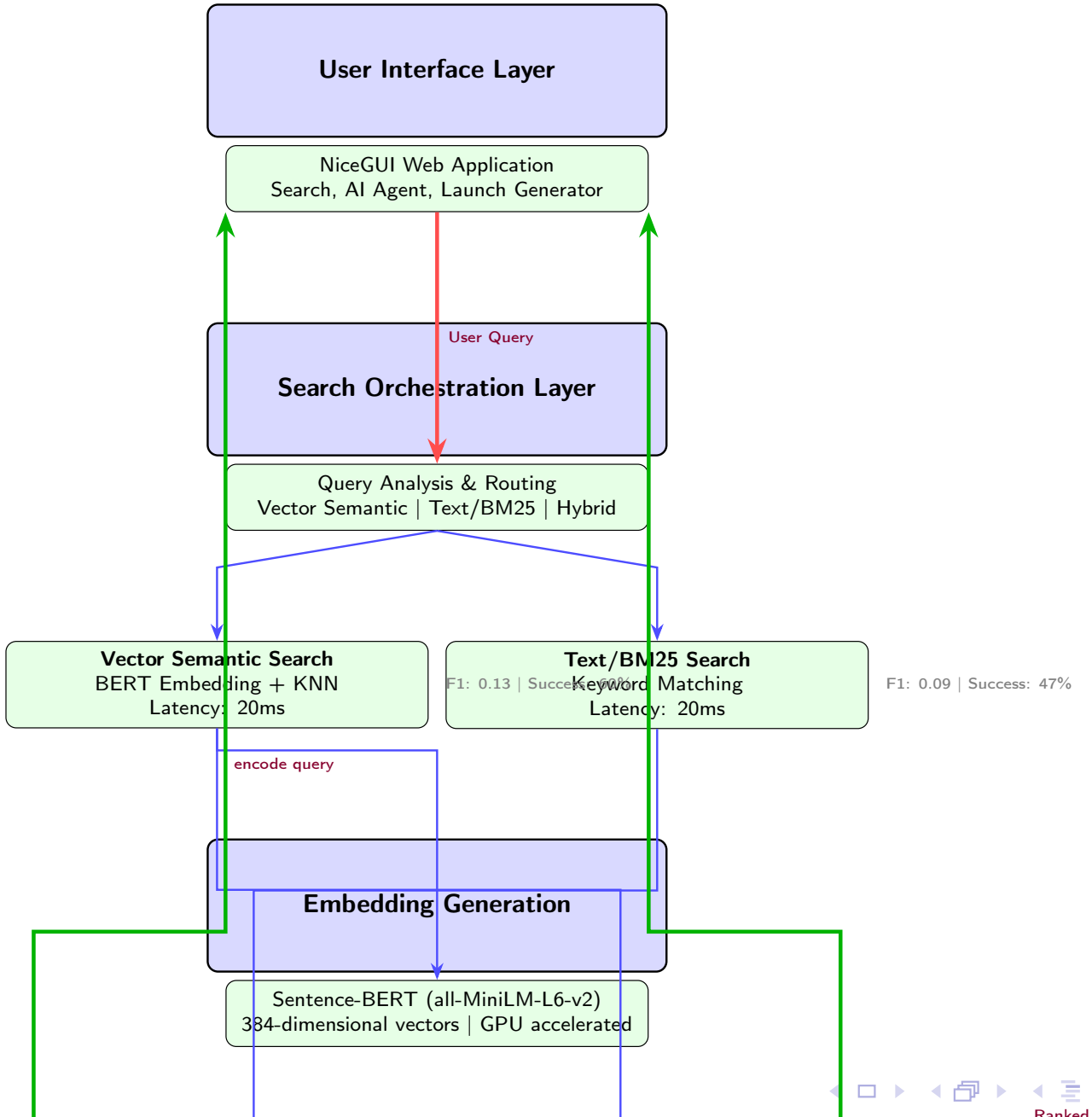
- **Project:** "Intelligent Asset Management for RAIL"
- **Aim:** Use robotics and AI to improve reliability and safety of European railway networks.
- **Challenge:** Railway operators lack ROS expertise, making it difficult to find and integrate the right software components for robotic inspection and maintenance systems.
- **Our Contribution:** The ROS Component Explorer was developed to accelerate this process by enabling rapid discovery and reuse of software components.

Research Contributions

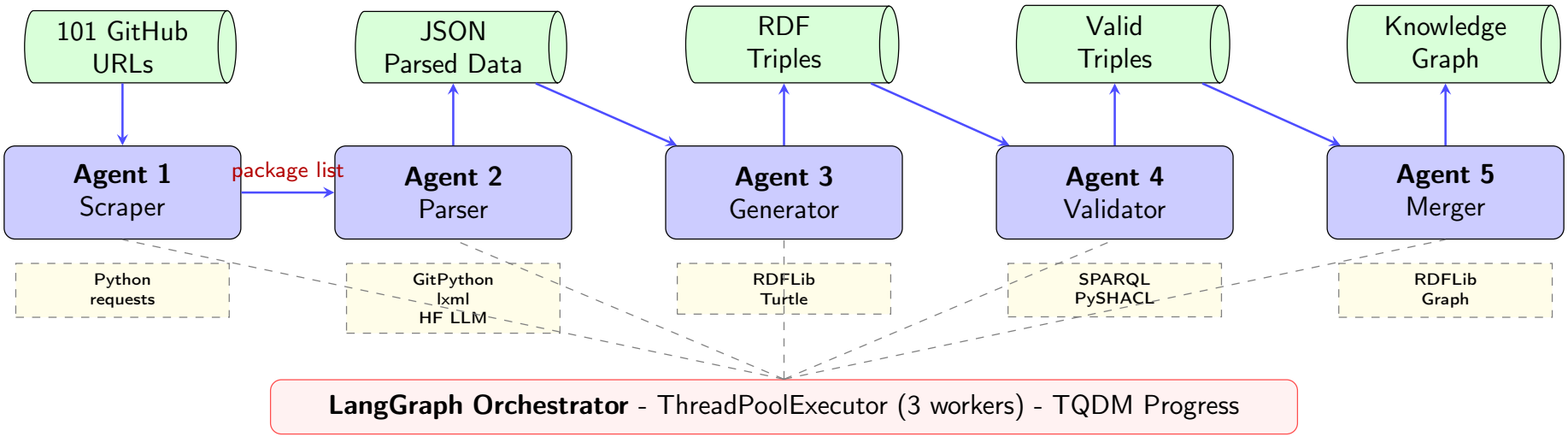
Key Contributions

- Analysis of limitations in keyword-based search for robotics software discovery
- A dual-layer semantic search architecture combining knowledge graphs and vector embeddings
- Empirical evaluation demonstrating improved retrieval performance over baseline methods

System Architecture: High-Level Overview



Knowledge Graph Generation Pipeline



Tech Stack & Data Flow

- **Agent 1:** Hardcoded list of 101 GitHub repos → **Agent 2:** GitPython clones repos, lxml parses package.xml, **HuggingFace LLM (openai/gpt-oss-20b)** extracts README insights
- **Agent 3:** RDFLib generates triples → **Agent 4:** PySHACL + SPARQL validates → **Agent 5:** Merges into final graph (4,410 triples)
- **Orchestration:** LangGraph StateGraph with ThreadPoolExecutor (parallel processing), TQDM for progress tracking

Layer 1: Structured Knowledge (RDF/OWL Ontology)

- **What it is:** A formal ontology and knowledge graph built with RDF/OWL.
- **How it works:** All data is stored as **RDF triples (Subject-Predicate-Object)**.
- **Strength:** Captures factual, structured relationships between components.

Real Example: `slam_toolbox` Package (from our KG)

Subject	Predicate	Object
ros:slam_toolbox	rdf:type	ros:LocalizationPackage
ros:slam_toolbox	rdfs:label	"Slam Toolbox"
ros:slam_toolbox	ros:packageVersion	"2.7.4"^^xsd:string
ros:slam_toolbox	ros:implementsAlgorithm	"Graph-based SLAM"
ros:slam_toolbox	ros:requiresHardware	hw:LaserScanner
ros:slam_toolbox	ros:runtimeDependsOn	ros:tf2
ros:slam_toolbox	ros:licenseType	"LGPL-2.1"^^xsd:string

Total: 4,410 triples across 101 packages with 15 predicate types

Layer 2: Semantic Understanding (Vector Embeddings)

- **What it is:** Dense vector representations of component descriptions.
- **The Problem:** The knowledge graph doesn't capture that "path planning" is semantically similar to "trajectory generation".
- **How it works:**
  1. Generate rich textual descriptions for each component (name + description + functionality + hardware).
  2. Use BERT-based sentence transformer (`all-MiniLM-L6-v2`) to convert text into 384-dimensional vectors.
  3. Store vectors in Apache Solr with HNSW indexing for efficient similarity search.
- **Strength:** Enables semantic similarity matching beyond exact keyword matches.

System Demonstration

Live Demo

Evaluation Overview

Example Queries from Evaluation Dataset

1. *"I'm using `nav2` for navigation. What localization packages are compatible?"*
2. *"My robot uses `gazebo_ros2_control` for simulation. What controller plugins work?"*
3. *"I use `ackermann_steering_controller`. What path planning algorithms support it?"*
4. *"I have RPLIDAR A1. What SLAM packages support this 2D LIDAR?"*
5. *"I have a RealSense D435 depth camera. What object detection packages work?"*
6. *"I want to replace cartographer with a faster SLAM solution. What are alternatives?"*

Challenge: These queries require semantic understanding - traditional keyword matching fails to capture user intent

Evaluation Overview

- Evaluated 4 search methods on 30 natural language ROS queries
- Dataset: Complex real-world scenarios (SLAM, navigation, perception, manipulation)
- Metrics: F1@10, NDCG@10, MAP, Latency

Evaluation Metrics

- **F1@10:** Harmonic mean of precision and recall in top-10 results
- **NDCG@10:** Normalized Discounted Cumulative Gain - measures ranking quality with position bias
- **MAP@10:** Mean Average Precision - average precision across all relevant results
- **Latency:** Average query response time in milliseconds

Evaluation Results: Performance Metrics

Metric	Keyword (BM25)	Vector Semantic	Hybrid (α=0.7)
F1@10	0.17	**0.19**	**0.19**
NDCG@10	0.35	**0.41**	**0.41**
MAP@10	0.27	**0.32**	0.31
Success@10	70.00%	**76.67%**	**76.67%**
Latency (ms)	27.32	**23.33**	62.90

- **Vector Semantic** and **Hybrid** search significantly outperformed the **Keyword** baseline on all accuracy and relevance metrics (F1, NDCG, MAP, Success@10).
- **Vector Semantic** is the clear winner, achieving the highest accuracy scores (or matching the Hybrid) while also having the **lowest latency** (23.33ms).

Summary

- The system's effectiveness comes from integrating two complementary layers:
  - **Layer 1 - Knowledge Graph (RDF/OWL):**
    - Provides structured, factual relationships
    - Enables precise filtering by type, hardware, functionality
  - **Layer 2 - Vector Embeddings (BERT):**
    - Captures semantic similarity and conceptual relationships
    - Handles natural language queries effectively
- **Hybrid retrieval** balances precision (structured data) with recall (semantic understanding)
- This novel contribution solves the "semantic gap"

Future Work & Research Directions

- **LLM-Assisted Query Understanding**
  - Integrate large language models to decompose complex queries
  - Enable multi-intent query handling (e.g., "SLAM + navigation + obstacle avoidance")
- **Automated Knowledge Graph Construction**
  - Extract semantic triples from ROS documentation automatically
  - Parse `package.xml`, READMEs, and source code
- **Domain-Specific Semantic Models**
  - Fine-tune BERT on ROS-specific corpus (e.g., BERT-ROS)
  - Improve semantic matching for robotics terminology

Thank You
Questions?